

INSTITUTO TECNOLÓGICO SUPERIOR DE
CALKINÍ, EN EL ESTADO DE CAMPECHE

ingeniería en Sistemas Computacionales

Nombre Completo & Matricula:

Fernando Jesus Colli Cach 9033

TOPICOS AVANZADOS DE PROGRAMACION

Maestro: JOSE MANUEL LUIS TURRIZA

Actividad No 3 Conexión a BD MySQL

4 SEMESTRE

GRUPO:A

REPORTE DE ACTIVIDAD PRÁCTICA No. 3

CONEXIÓN ENTRE JAVA Y MYSQL

Sistema de Ventas - Ferretería

1. OBJETIVOS

1.1 Objetivo General

Implementar una aplicación de escritorio en Java que establezca una conexión funcional con una base de datos MySQL, permitiendo consultar y visualizar información de ventas asociadas a empleados a través de una interfaz gráfica desarrollada con JavaFX.

1.2 Objetivos Específicos

- Establecer una conexión exitosa entre Java y MySQL mediante el conector JDBC.
- Diseñar una arquitectura por capas (Modelo-Vista-Controlador) para organizar el código.
- Crear una interfaz gráfica que muestre los datos obtenidos de la base de datos en una tabla.
- Consultar mediante sentencias SQL la información combinada de las tablas ventas y empleados.
- Emplear Maven como gestor de dependencias para facilitar la configuración del proyecto.

2. DESCRIPCIÓN DEL PROBLEMA

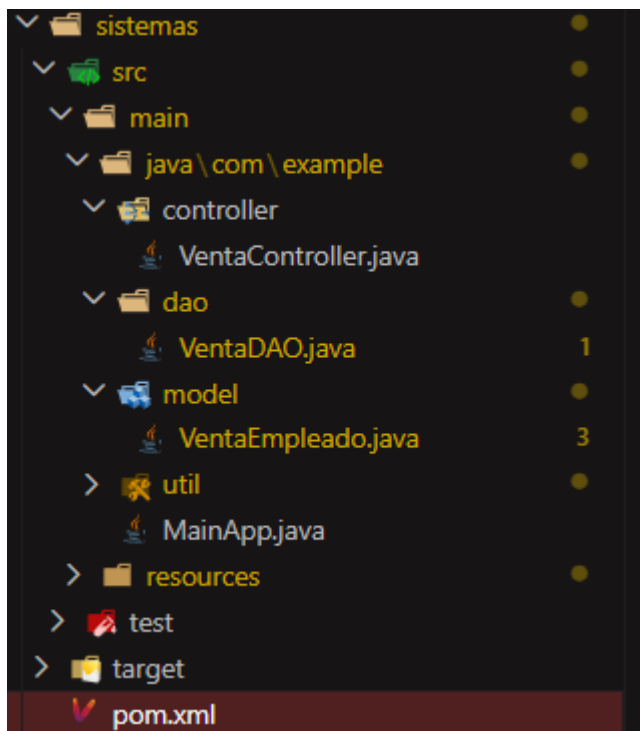
Se requiere desarrollar un sistema de consulta para una ferretería que permita visualizar el historial de ventas realizadas, mostrando por cada venta: el identificador de la venta (ID Venta), el nombre del empleado que la realizó y el monto total de la misma.

La información se encuentra almacenada en una base de datos MySQL con dos tablas relacionadas: ventas (que contiene idVenta, total y idEmpleado como clave foránea) y empleados (que contiene idEmpleado y nombre). La aplicación debe recuperar estos datos mediante una combinación de ambas tablas y presentarlos al usuario a través de una interfaz amigable.

3. METODOLOGÍA Y DESARROLLO

3.1 Estructura del Proyecto

Se implementó una arquitectura multicapa siguiendo el patrón MVC (Modelo-Vista-Controlador). La estructura de directorios del proyecto es la siguiente:



3.2 Configuración de Dependencias (pom.xml)

Se utilizó Maven para gestionar las dependencias del proyecto. Las principales configuraciones fueron:

```
pom.xml x
sistemas > pom.xml
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5      <modelVersion>4.0.0</modelVersion>
6
7      <groupId>com.sistemas</groupId>
8      <artifactId>sistemas</artifactId>
9      <version>1.0-SNAPSHOT</version>
10
11     <properties>
12         <maven.compiler.source>17</maven.compiler.source>
13         <maven.compiler.target>17</maven.compiler.target>
14         <javafx.version>17.0.2</javafx.version>
15     </properties>
16
17     <dependencies>
18         <dependency>
19             <groupId>org.openjfx</groupId>
20             <artifactId>javafx-controls</artifactId>
21             <version>${javafx.version}</version>
22         </dependency>
23         <dependency>
24             <groupId>org.openjfx</groupId>
25             <artifactId>javafx-fxml</artifactId>
26             <version>${javafx.version}</version>
27         </dependency>
28         <dependency>
29             <groupId>com.mysql</groupId>
30             <artifactId>mysql-connector-j</artifactId>
31             <version>8.0.33</version>
32         </dependency>
33     </dependencies>
34
35     <build>
36         <plugins>
37             <plugin>
38                 <groupId>org.openjfx</groupId>
39                 <artifactId>javafx-maven-plugin</artifactId>
40                 <version>0.0.8</version>
41                 <configuration>
42                     <mainClass>com.example.MainApp</mainClass>
43                 </configuration>
44             </plugin>
45         </plugins>
46     </build>
47 </project>
```

sistemas > pom.xml

```
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
8   <artifactId>sistemas</artifactId>
9   <version>1.0-SNAPSHOT</version>
10
11   <properties>
12     <maven.compiler.source>17</maven.compiler.source>
13     <maven.compiler.target>17</maven.compiler.target>
14     <javafx.version>17.0.2</javafx.version>
15
16   </properties>
17
18   <dependencies>
19     <dependency>
20       <groupId>org.openjfx</groupId>
21       <artifactId>javafx-controls</artifactId>
22       <version>${javafx.version}</version>
23
24     </dependency>
25     <dependency>
26       <groupId>org.openjfx</groupId>
27       <artifactId>javafx-fxml</artifactId>
28       <version>${javafx.version}</version>
29     </dependency>
30     <dependency>
31       <groupId>com.mysql</groupId>
32       <artifactId>mysql-connector-j</artifactId>
33       <version>8.0.33</version>
34     </dependency>
35   </dependencies>
36
37   <build>
38     <plugins>
39       <plugin>
40         <groupId>org.openjfx</groupId>
41         <artifactId>javafx-maven-plugin</artifactId>
42         <version>0.0.8</version>
43         <configuration>
44           <mainClass>com.example.MainApp</mainClass>
45         </configuration>
46       </plugin>
47     </plugins>
48   </build>
49 </project>
```

3.3 Capa de Modelo (VentaEmpleado.java)

Se creó la clase VentaEmpleado como POJO (Plain Old Java Object) para representar cada registro de venta:

Atributo	Tipo	Descripción
idVenta	String	Identificador único de la venta
nombreEmpleado	String	Nombre del empleado que realizó la venta
total	double	Monto total de la venta

La clase incluye constructor, getters y (opcionalmente setters) para acceder a los datos desde otras capas.

3.4 Capa de Acceso a Datos (VentaDAO.java)

En esta clase se implementó la lógica de consulta a la base de datos mediante JDBC:

```
VentaDAO.java 1 X
sistemas > src > main > java > com > example > dao > VentaDAO.java > ...
1  package com.example.dao;
2
3  import java.sql.Connection;
4  import java.sql.ResultSet;
5  import java.sql.Statement;
6  import java.util.ArrayList;
7  import java.util.List;
8
9  import com.example.model.VentaEmpleado;
10 import com.example.util.ConexionBD;
11
12 public class VentaDAO {
13
14     public static List<VentaEmpleado> obtenerVentas() {
15         List<VentaEmpleado> lista = new ArrayList<>();
16
17         String sql = "SELECT v.idVenta, e.nombre, v.total FROM ventas v JOIN empleados e ON v.idEmpleado = e.idEmpleado";
18
19         try (Connection con = ConexionBD.conectar(); Statement st = con.createStatement(); ResultSet rs = st.executeQuery(sql)) {
20
21             while (rs.next()) {
22                 VentaEmpleado venta = new VentaEmpleado(
23                     rs.getString(columnLabel: "idVenta"),
24                     rs.getString(columnLabel: "nombre"),
25                     rs.getDouble(columnLabel: "total")
26                 );
27                 lista.add(venta);
28             }
29         } catch (Exception e) {
30             e.printStackTrace();
31         }
32
33         return lista;
34     }
35 }
36
```

Nota técnica: Se utilizó la sentencia JOIN para combinar las tablas ventas y empleados mediante la clave foránea idEmpleado.

3.5 Capa de Controlador (VentaController.java)

El controlador gestiona la lógica de presentación: vincula los datos obtenidos del DAO con los componentes visuales definidos en el archivo FXML.

```
VentaController.java X
sistemas > src > main > java > com > example > controller > VentaController.java > ...
1 package com.example.controller;
2
3 import com.example.dao.VentaDAO;
4 import com.example.model.VentaEmpleado;
5
6 import javafx.collections.FXCollections;
7 import javafx.fxml.FXML;
8 import javafx.scene.control.TableColumn;
9 import javafx.scene.control.TableView;
10 import javafx.scene.control.cell.PropertyValueFactory;
11
12 public class VentaController {
13
14     @FXML
15     private TableView<VentaEmpleado> tablaVentas;
16
17     @FXML
18     private TableColumn<VentaEmpleado, Integer> colId;
19
20     @FXML
21     private TableColumn<VentaEmpleado, String> colEmpleado;
22
23     @FXML
24     private TableColumn<VentaEmpleado, Double> colTotal;
25
26     @FXML
27     public void initialize() {
28         colId.setCellValueFactory(new PropertyValueFactory<>(property: "idVenta"));
29         colEmpleado.setCellValueFactory(new PropertyValueFactory<>(property: "nombreEmpleado")); // ← CAMBIADO
30         colTotal.setCellValueFactory(new PropertyValueFactory<>(property: "total"));
31
32         tablaVentas.setItems(FXCollections.observableArrayList(VentaDAO.obtenerVentas()));
33     }
34 }
35
```

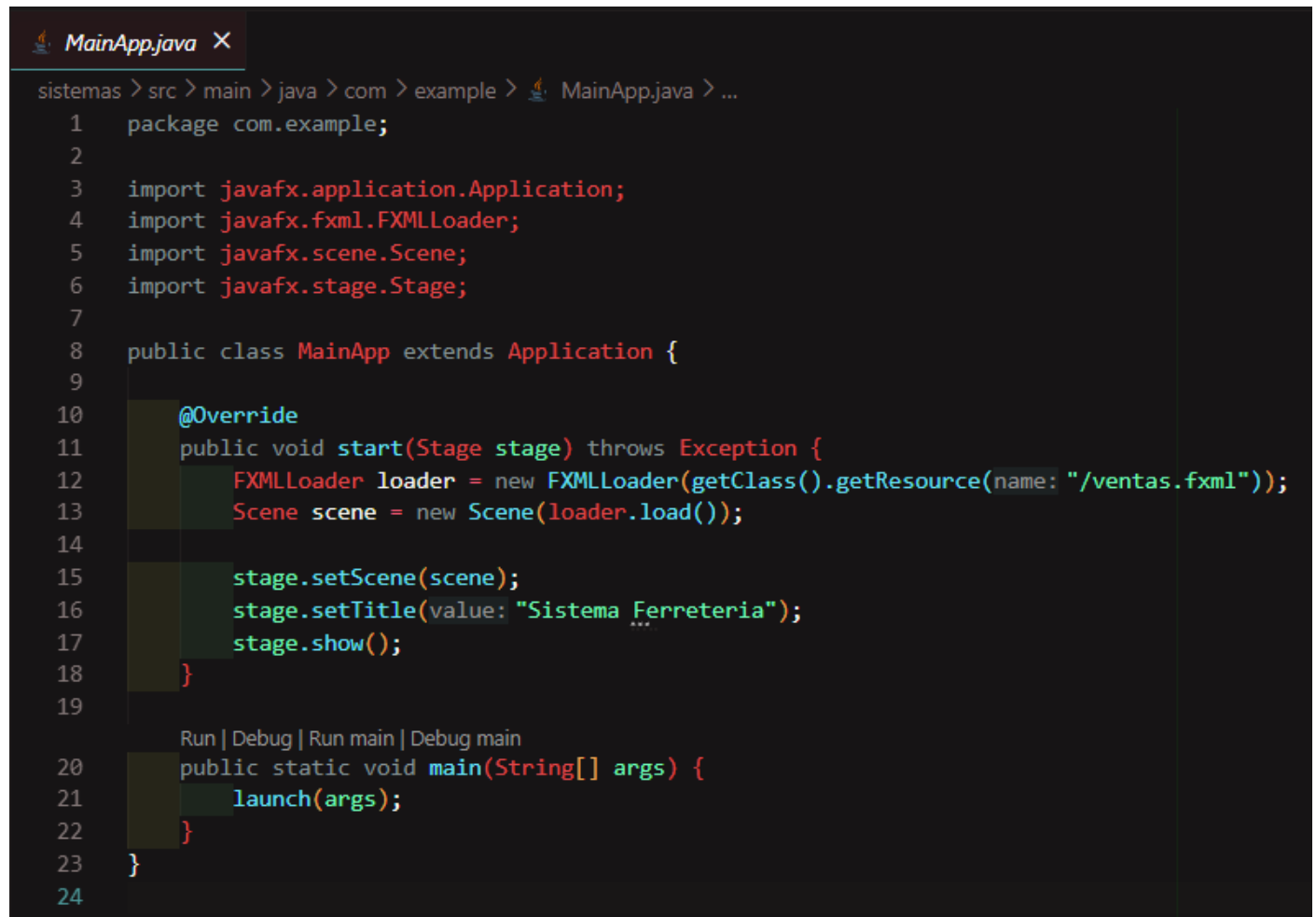
3.6 Capa de Utilería (ConexionBD.java)

Se implementó una clase utilitaria con un método estático conectar() que retorna un objeto Connection a la base de datos MySQL. Los parámetros de conexión (URL, usuario, contraseña) se configuran internamente.

```
try (Connection con = ConexionBD.conectar(); Statement st = con.createStatement(); ResultSet rs = st.executeQuery(sql)) {
```

3.7 Aplicación Principal (MainApp.java)

Clase que extiende Application de JavaFX y lanza la interfaz gráfica cargando el archivo ventas.fxml:



```
1 package com.example;
2
3 import javafx.application.Application;
4 import javafx.fxml.FXMLLoader;
5 import javafx.scene.Scene;
6 import javafx.stage.Stage;
7
8 public class MainApp extends Application {
9
10     @Override
11     public void start(Stage stage) throws Exception {
12         FXMLLoader loader = new FXMLLoader(getClass().getResource(name: "/ventas.fxml"));
13         Scene scene = new Scene(loader.load());
14
15         stage.setScene(scene);
16         stage.setTitle(value: "Sistema Ferretería");
17         stage.show();
18     }
19
20     Run | Debug | Run main | Debug main
21     public static void main(String[] args) {
22         launch(args);
23     }
24 }
```

4. RESULTADOS OBTENIDOS

4.1 Interfaz Gráfica Final

La aplicación desplegó exitosamente una tabla con los siguientes datos provenientes de la base de datos MySQL:

The screenshot shows a Java IDE with the following components:

- EXPLORADOR:** A file explorer on the left showing a project structure with folders like 'src', 'main', 'controller', 'dao', 'model', 'util', 'resources', 'test', 'target', and 'pom.xml'. The file 'MainApp.java' is selected.
- MainApp.java:** The source code of the application, which is a JavaFX application. It imports 'javafx.application.Application', 'javafx.fxml.FXMLLoader', 'javafx.scene.Scene', and 'javafx.stage.Stage'. The code defines a 'MainApp' class that extends 'Application' and overrides the 'start' method to load an FXML file named 'ventas.fxml' and set the scene.
- Sistema Ferreteria:** A running application window titled 'Sistema Ferreteria' displaying a table titled 'Sistema de Ventas - Ferretería'. The table has three columns: 'ID Venta', 'Empleado', and 'Total'. It contains three rows of data:

ID Venta	Empleado	Total
1	Juan Pérez	1500.0
2	Juan Pérez	2300.5
3	María López	3200.0

The IDE status bar at the bottom shows 'Lín. 24, col. 1', 'Espacios: 4', 'UTF-8', 'LF', 'Java', 'Go Live', 'Background', 'Prettier', and 'Zero Two'.

4.2 Funcionalidades Verificadas

Función	Estado	Observación
Conexión a MySQL	Exitosa	El conector JDBC 8.0.33 funcionó correctamente
Consulta JOIN entre tablas	Exitosa	Se combinaron ventas y empleados
Visualización en TableView	Exitosa	Los datos se mostraron correctamente
Carga automática al iniciar	Exitosa	initialize() ejecuta la consulta al abrir la ventana

MySQL Workbench

Local instance MySQL94 - W...

File Edit View Query Database Server Tools Scripting Help

ferreteria

Limit to 10000 rows

```
3 • USE ferreteria;
4
5 -- Crear tabla empleados
6 • CREATE TABLE IF NOT EXISTS empleados (
7     idEmpleado INT PRIMARY KEY AUTO_INCREMENT,
8     nombre VARCHAR(100)
9 );
10
```

Result Grid

	idVenta	nombre	total
▶	1	Juan Pérez	1500.00
	2	Juan Pérez	2300.50
	3	María López	3200.00

Result 1

Read Only

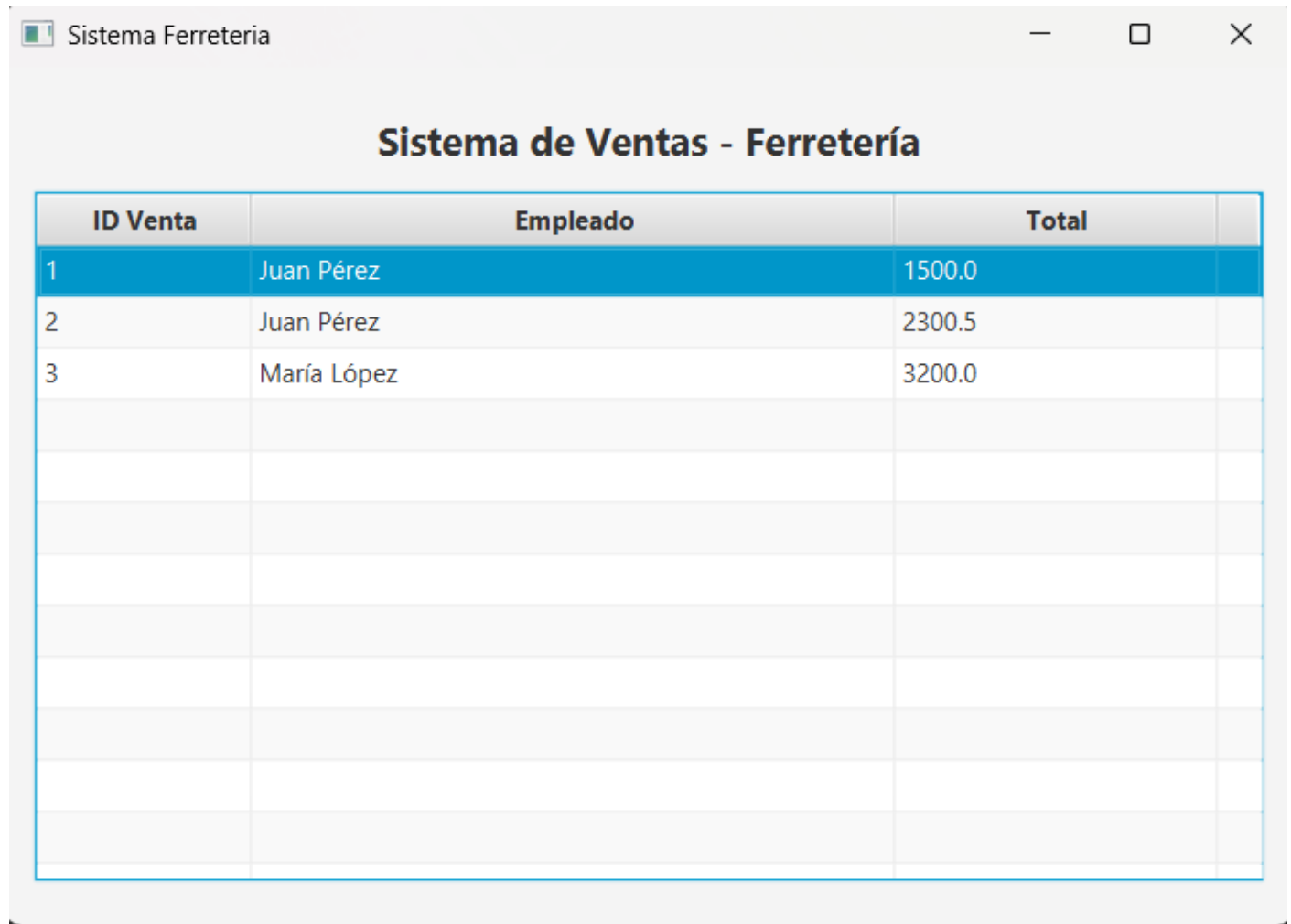
Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 1	14:32:55	CREATE DATABASE IF NOT EXISTS ferreteria	1 row(s) affected	0.016 sec
✓ 2	14:32:55	USE ferreteria	0 row(s) affected	0.000 sec
✓ 3	14:32:55	CREATE TABLE IF NOT EXISTS empleados (idEmpleado INT PRIMARY K...	0 row(s) affected	0.047 sec
✓ 4	14:32:55	CREATE TABLE IF NOT EXISTS ventas (idVenta INT PRIMARY KEY AUT...	0 row(s) affected	0.031 sec
✓ 5	14:32:55	INSERT INTO empleados (nombre) VALUES ('Juan Pérez'), ('María López')	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.016 sec
✓ 6	14:32:55	INSERT INTO ventas (idEmpleado, total) VALUES (1, 1500.00), (1, 2300.50), (...)	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.000 sec
✓ 7	14:32:55	SELECT v.idVenta, e.nombre, v.total FROM ventas v JOIN empleados e ON v.i...	3 row(s) returned	0.000 sec / 0.000 sec

4.3 Evidencia de Ejecución

Se muestra a continuación la interfaz resultante del sistema:



ID Venta	Empleado	Total	
1	Juan Pérez	1500.0	
2	Juan Pérez	2300.5	
3	María López	3200.0	

5. ANÁLISIS Y DISCUSIÓN

5.1 Interpretación de Resultados

La aplicación logró cumplir con todos los objetivos planteados. La arquitectura MVC permitió una clara separación de responsabilidades:

- **Modelo** (VentaEmpleado): Define la estructura de los datos.
- **Vista** (ventas.fxml): Define la interfaz gráfica.
- **Controlador** (VentaController): Conecta la vista con los datos.
- **DAO** (VentaDAO): Encapsula el acceso a la base de datos.

5.2 Dificultades Encontradas y Soluciones

Dificultad	Solución Aplicada
Configuración de JavaFX en Maven	Se agregaron las dependencias correctas en pom.xml con la versión 17.0.2
Error de tipo de dato en columna idVenta	Se declaró como String en el modelo para mayor flexibilidad
El plugin javafx-maven-plugin no encontraba la MainClass	Se especificó correctamente com.example.MainApp en la configuración
Nombre de propiedad incorrecto en PropertyValueFactory	Se corrigió nombreEmpleado (coincidiendo con el getter getNombreEmpleado())

5.3 Limitaciones del Sistema Actual

- **Funcionalidades limitadas:** Solo consulta datos; no permite insertar, actualizar o eliminar registros.
- **Manejo de errores básico:** Las excepciones solo se imprimen en consola sin retroalimentación al usuario.
- **Configuración de conexión fija:** Los parámetros de conexión están hardcodeados en ConexionBD.java.

CONCLUSION

La actividad práctica permitió implementar exitosamente una aplicación en JavaFX que se conecta a una base de datos MySQL para consultar y visualizar información de ventas de una ferretería. Se logró establecer la conexión mediante JDBC, aplicar el patrón MVC para organizar el código y mostrar los datos en una interfaz gráfica de tabla. El proyecto cumple con los objetivos planteados, demostrando la integración funcional entre Java y MySQL como solución para sistemas de consulta de información.

REFERENCIAS

1. Oracle Corporation. (2021). *JavaFX Documentation*. Recuperado de <https://openjfx.io>
2. MySQL. (2023). *MySQL Connector/J Developer Guide*. Recuperado de <https://dev.mysql.com/doc/connector-j/en/>
3. Apache Maven. (2023). *Maven Getting Started Guide*. Recuperado de <https://maven.apache.org/guides/getting-started/>
4. Schildt, H. (2019). *Java: The Complete Reference* (11th ed.). McGraw-Hill.